

Universidad Distrital Francisco José de Caldas
Department of Systems Engineering
Faculty of Engineering
Programming for Data Analysis



Project Definition and Source Research

Jefferson David Ortiz Buitrago – Code: 20221020110
Cristian Andres Cruz Puentes – Code: 20221020125
Esteban Alexander Bautista Solano – Code: 20221020089

1. Executive Summary

This document presents the improvements, governance metrics, storytelling aggregations, and Gold layer implementation for the **Real Madrid Pulse** data engineering project.

The pipeline integrates two data sources: web scraping from `football-espana.net` and Reddit API ingestion via the `r/realmadrid` subreddit. Both are orchestrated through Apache Airflow DAGs running inside a Docker Compose environment, following a **Medallion Architecture** (Bronze → Silver → Gold).

The Bronze layer captures raw data in JSON/CSV format. The Silver layer applies schema enforcement, deduplication, text normalization, and outlier treatment using IQR-based capping, persisting results in both PostgreSQL and Parquet. The Gold layer reads all Silver Parquet files using PySpark, runs a full NLP pipeline with spaCy and VADER sentiment analysis, and produces governance KPIs plus storytelling aggregations as independent Parquet files.

The final processed dataset consists of **280 records** (200 from scraping, 80 from Reddit), with a global positive sentiment rate of **56.43%** and an average VADER compound score of **0.2068**. All governance KPIs report a schema compliance rate of 100%, confirming the integrity of the pipeline end-to-end.

2. Pipeline Improvements

2.1 Web Scraping Silver DAG — Changes and Justification

The Silver DAG for web scraping (`football_silver_pipeline`) was substantially improved to support better data quality, reproducibility, and analytical consistency. The DAG reads bronze files from

/opt/airflow/datalake_bronze/, detects only new files using an Airflow Variable (last_processed_bronze_ts) for timestamp-based idempotency, and applies the following transformations:

- **Incremental processing via timestamp idempotency:** The task `detect_new_files()` reads the Airflow Variable `last_processed_bronze_ts` and filters only files with a timestamp greater than the last run. This prevents reprocessing and ensures exactly-once semantics across DAG runs.
- **Enhanced text cleaning and HTML noise filtering:** The `clean_text_basic()` function removes URLs, typographic quotes, punctuation, and extra whitespace. Additionally, paragraph extraction uses a noise phrase filter that discards boilerplate HTML fragments ("Leave a Reply", "Cancel reply", etc.) and paragraphs shorter than 80 characters. Lemmatization is intentionally deferred to the Gold layer to keep the Silver DAG lightweight.
- **Explicit schema enforcement before PostgreSQL insertion:** A `SILVER_SCHEMA` list defines the exact 11 columns (`url`, `title`, `author`, `published_at`, `reading_time_min`, `body_text`, `body_text_clean`, `title_clean`, `body_text_missing`, `bronze_source`, `source`) that are selected from the DataFrame before any persistence step. Columns outside this schema are dropped, preventing unexpected fields from propagating downstream.
- **Dual persistence into PostgreSQL and Parquet:** The `save_to_db()` task first inserts records into the `articles_scraping` table using `ON CONFLICT (url) DO NOTHING` to enforce URL uniqueness at the database level. It then writes the same DataFrame to a timestamped Parquet file under `/opt/airflow/datalake_silver/`. This dual write provides both queryable relational storage and efficient columnar storage for PySpark consumption in the Gold layer.
- **URL-based deduplication with SQL constraints:** Before schema enforcement, `drop_duplicates (subset=["url"])` removes in-memory duplicates. The PostgreSQL `ON CONFLICT` clause serves as a second deduplication barrier. This two-layer strategy ensures idempotency even if the same bronze file is accidentally reprocessed.

An additional improvement is the IQR-based outlier treatment applied to `reading_time_min`: values below $Q_1 - 1.5 \cdot IQR$ or above $Q_3 + 1.5 \cdot IQR$ are clipped to the fence values. For the latest batch, $Q_1 = 2.0$, $Q_3 = 3.0$, $IQR = 1.0$, giving bounds $[0.5, 4.5]$. Six records (6%) were clipped. Before and after boxplots are saved automatically to `datalake_silver/outlier_reports/`.

2.2 Reddit API Silver DAG — Changes and Justification

The Reddit API Silver DAG (`reddit_api_silver_pipeline`) was improved through the following changes:

- **Handling deleted or removed posts:** Reddit posts where `selftext` equals "[deleted]" or

"[removed]" are treated as missing body text. The field is set to an empty string and flagged via `body_text_missing = True`. This prevents corrupted text from flowing into the NLP pipeline.

- **UTC datetime normalization:** The field `created_utc` (Unix timestamp) is converted to a timezone-aware datetime using `pd.to_datetime(df["created_utc"], unit="s", utc=True)`, producing consistent ISO 8601 timestamps compatible with the scraping source.
- **Validation of engagement metrics:** Both `score` and `num_comments` are coerced to float with `pd.to_numeric(..., errors="coerce")` before IQR treatment. This handles any non-numeric values that the Reddit API may return for deleted or shadow-banned posts.
- **Text normalization and cleaning:** The `clean_text_basic()` function for Reddit additionally removes hashtags (`#word`), user mentions (`u/user`), and subreddit references (`r/name`), which are common in Reddit text but irrelevant to sentiment analysis.
- **PostgreSQL + Parquet dual persistence:** Identical dual-write strategy as the web scraping DAG. The `reddit_posts` table uses `ON CONFLICT (url) DO NOTHING` for deduplication. IQR outlier capping is applied independently to `score` and `num_comments`, with plots saved to `outlier_reports/`.

For the latest Reddit batch (80 posts): median `score` = 4.0, median `num_comments` = 2.0, and 39 posts (48.75%) had missing body text, confirming that title-only posts are frequent in this subreddit.

2.3 Before/After Comparison Table

Table 1. Pipeline Improvements Comparison

Aspect	Before	After
Incremental Processing	All bronze files reprocessed on every run	Timestamp-based idempotency via Airflow Variable; only new files processed
Storage	PostgreSQL only	PostgreSQL + Parquet (dual write per run)
Schema Validation	Partial; extra columns passed downstream	Explicit SILVER_SCHEMA list; columns outside schema dropped before insert
Text Cleaning	Basic lowercasing and URL removal	Advanced: HTML boilerplate filter, noise phrase removal, Reddit-specific patterns (hashtags, mentions, subreddits)
Deduplication	In-memory only	Two-layer: in-memory drop_duplicates + SQL ON CONFLICT
Outlier Treatment	None	IQR capping on reading_time_min (scraping) and score/num_comments (Reddit), with auto-generated boxplot reports
Null Handling	Implicit (pandas NaN)	Explicit strategies per field: sentinel “Unknown” for authors, median imputation for reading_time_min, boolean flag for missing body text

2.4 Gold Layer NLP Refinement — Stopword Tuning

After inspecting the first keyword outputs, the spaCy preprocessing in the Gold DAG was refined to remove noise observed in the real corpus, addressing the Workshop requirement to *adjust stop word lists and*

domain-specific noise removal:

- **Spanish stopwords leakage:** The spaCy model is English-only (`en_core_web_sm`), so Spanish function words (“el”, “la”, “los”...) leaking from *El Clásico*, *La Liga*, and quoted Spanish text were not filtered. A curated Spanish stopwords set was added.
- **Lemma re-check:** Each token’s *lemma* is now re-checked against the spaCy stopwords list, catching inflected forms (e.g. “going” → “go”) that slipped through the original surface-form check.
- **Minimum token length and domain noise:** Tokens shorter than 3 characters are dropped, and an expanded domain stopwords list removes generic football/time terms (“week”, “goal”, “time”...) that do not explain sentiment, while sentiment-bearing words (“good”, “win”...) are intentionally preserved.

3. Data Quality Findings

3.1 Web Scraping Source — Observed Issues

Several data quality issues were identified during Silver processing of the `football-espana.net` scraping source:

- **Missing authors:** Many articles lacked an author meta tag. These were imputed with the sentinel value “Unknown” to maintain a non-null field without introducing fabricated data.
- **Empty article bodies:** Articles where all paragraphs were shorter than 80 characters or consisted entirely of boilerplate phrases (e.g., “*Leave a Reply*”, “*Comment**”) produced a null `body_text`. These were flagged via the boolean field `body_text_missing`. In the processed Silver dataset, 13.93% of all records had missing body text.
- **HTML boilerplate text:** Raw paragraph extraction frequently included comment form prompts, cookie notices, and navigation labels. A `NOISE_PHRASES` filter list with 13 patterns was applied during extraction to discard these fragments.
- **Duplicate URLs:** Multiple bronze files captured the same articles from overlapping pagination. The two-layer deduplication strategy (in-memory + SQL) removed these duplicates, with the metrics file reporting 0 duplicates in the most recent batch due to prior incremental filtering.
- **Inconsistent publication dates:** Some articles had no `article:published_time` meta tag. A fallback mechanism parsed `published_time` using `dayfirst=True` for European date formats, recovering dates for affected records.

3.2 Reddit API Source — Observed Issues

The Reddit API source (r/realmadrid) presented the following issues:

- **Deleted and removed posts:** Posts where `selftext` contained "[deleted]" or "[removed]" were common. These were treated as missing body text and flagged. In the last ingested batch, 39 out of 80 posts (48.75%) had missing body text.
- **Missing selftext fields:** Title-only posts (i.e., link posts without body text) returned null or empty `selftext`. The VADER fallback uses the `title` field for sentiment scoring in these cases.
- **Informal language and emojis:** Reddit posts contained heavy informal language, subreddit-specific slang (u/user, r/subreddit), and emojis that interfere with tokenization. The enhanced `clean_text_basic()` function removes these patterns before NLP processing.
- **High variability in text length:** Body text length varied from 0 to several thousand characters. The median text length for Reddit in the Gold layer was only 10 characters (reflecting the many empty-body posts), compared to 1,821 characters for the scraping source.

3.3 Descriptive Statistics Tables

The following table summarizes the key statistics computed from the final Gold layer dataset (280 total records):

Table 2. Descriptive Statistics — Gold Layer (280 records)

Metric	Web Scraping	Reddit	Global
Total Records	200	80	280
Body Text Missing Rate	—	48.75%	13.93%
Avg. Text Length (chars)	1,976	1,168	1,746
Median Text Length (chars)	1,821	10	1,727
Avg. Token Count (NLP)	146	101	133
VADER Compound Mean	0.2324	0.1428	0.2068
VADER Compound Std	0.7411	0.5501	0.6922
Positive Sentiment %	62.0%	42.5%	56.43%
Neutral Sentiment %	6.0%	26.25%	11.79%
Negative Sentiment %	32.0%	31.25%	31.79%
Schema Compliance Rate	—	—	100%
Null Rate (all key fields)	—	—	0%

3.4 Visualizations (Outlier Boxplots, Null Rate Charts, Text Length Distributions)

The Silver processing pipeline automatically generates the following visualizations for each bronze file processed, stored in `datalake_silver/outlier_reports/`:

- **Outlier boxplots:** Side-by-side box plots for each numeric field before and after IQR capping. For web scraping, these cover `reading_time_min`. For Reddit, separate plots are generated for `score` and `num_comments`. The last web scraping batch detected 6 outliers (6%) in `reading_time_min`, with $Q_1 = 2.0$, $Q_3 = 3.0$, bounds $[0.5, 4.5]$.
- **Null rate bar charts:** Generated in the exploratory notebooks (`notebook_1_data_quality_findings.ipynb`), showing null rates per column for both sources. The Gold governance Parquet confirms 0% null rate for all key fields after Silver cleaning.
- **Text length distributions:** Histogram comparing `body_text_clean` character counts between sources. The scraping source shows a near-normal distribution centered around 1,821 characters (median), while the Reddit source is heavily right-skewed due to the large proportion of title-only posts.
- **Sentiment polarity charts:** Generated from Gold aggregations; see Section 5.

4. Governance KPI Definition

Governance KPIs are computed in Gold DAG Task 4 (`save_gold_and_governance`) and persisted as `governance_YYYYMMDD_HHMMSS.parquet`. The last run (20260529_215404) produced the following KPIs:

4.1 Null Rate per Field

Measures the percentage of missing values per key column across the full dataset.

$$\text{Null Rate}_{col} = \frac{\text{Null Values in } col}{\text{Total Records}} \times 100$$

All 9 key fields (`url`, `title`, `author`, `published_at`, `body_text`, `body_text_clean`, `source`, `vader_compound`, `text_processed`) reported a null rate of **0.0%** in the last pipeline run, confirming the effectiveness of the Silver null-handling strategies (sentinel values, fallback dates, median imputation).

4.2 Volume Metrics

Tracks the total number of records ingested and their distribution by source per pipeline run.

- `total_records`: 280
- `records_reddit`: 80
- `records_scraping`: 200

Volume is also tracked temporally via the `volume_trend` storytelling aggregation (Agg 6), which shows weekly record counts per source. The scraping source shows decreasing volume from 60 records (week of 2026-05-04) to 36 records (week of 2026-05-25), while Reddit contributed 80 records in the final week.

4.3 Duplicate Rate

Measures the number of records removed by the global cross-source deduplication step in Task 1 of the Gold DAG (`df.dropDuplicates(["url"])`). This complements the Silver-level deduplication and covers cases where the same URL appears in both a scraping and a Reddit post (e.g., shared article links).

$$\text{Duplicate Rate} = \frac{\text{Records Removed by dropDuplicates}}{\text{Records Before Dedup}} \times 100$$

In the last run, no cross-source URL duplicates were found, reflecting the distinct nature of the two data sources.

4.4 Schema Compliance Rate

Measures the percentage of records where all required fields (`url`, `title`, `author`, `published_at`, `body_text`, `source`) are non-null simultaneously.

$$\text{Schema Compliance Rate} = \frac{\text{Records with all required fields non-null}}{\text{Total Records}} \times 100$$

The last run reported a **schema compliance rate of 100%**, confirming that all records entering the Gold layer meet the minimum required schema defined in the Silver DAGs.

4.5 Outlier Rate per Numeric Field

Detects the residual percentage of values still outside the IQR bounds after Silver capping. This is used as a post-treatment validation signal.

- `outlier_rate_reading_time_min` (scraping): **0.0%** — Silver capping was effective; no residual outliers remain. IQR bounds: [2.0, 2.0].
- `outlier_rate_score` (Reddit): **0.0%** — Score distribution was within bounds after capping. IQR bounds: [-26.0, 46.0].
- `outlier_rate_num_comments` (Reddit): **0.0%** — No residual outliers. IQR bounds: [-21.75, 36.25].

The 0% residual outlier rate validates the IQR capping applied in the Silver layer.

4.6 Text Length Statistics

Tracks minimum, maximum, mean, and median character length of `body_text_clean` globally and per source, providing a signal for detecting anomalous ingestion batches (e.g., a run producing only title-only posts would show an abnormally low mean text length).

- Global mean: 1,745.56 chars | Median: 1,727.0 | Min: 0 | Max: 29,084
- Scraping: mean = 1,976.46 | median = 1,821.0
- Reddit: mean = 1,168.31 | median = 10.0

The large gap between Reddit mean and median (1,168 vs. 10) confirms the highly skewed distribution caused by empty-body posts and justifies using median as the primary text-length governance signal for Reddit.

4.7 Ingestion Frequency Compliance

Measures whether the actual number of ingestion runs matches the expected schedule. With a @daily schedule, 7 runs are expected per week.

$$\text{Compliance}_{\text{source}} = \min\left(\frac{\text{Actual Runs}}{\text{Expected Runs}} \times 100, 100\right)$$

In the current dataset, only one distinct `bronze_source` file was processed per source, yielding a compliance of **14.29%** for both scraping and Reddit. This reflects the project development phase; a production deployment running for a full week would reach 100%.

4.8 Sentiment Distribution as Governance Signal

Unexpected sentiment distributions can indicate ingestion anomalies. A sudden shift toward 90%+ negative sentiment, for example, could signal that a scraper started capturing comment sections or error pages instead of article bodies. The governance Parquet tracks:

- `sentiment_pct_positive`: 56.43%
- `sentiment_pct_neutral`: 11.79%
- `sentiment_pct_negative`: 31.79%
- `compound_mean`: 0.2068 | `compound_std`: 0.6922

The high standard deviation (0.6922) reflects the mixed content of the dataset, covering topics ranging from transfer announcements (positive) to injury reports (negative). The distribution is consistent across runs and represents a stable governance baseline.

5. Storytelling Aggregations

5.1 Dashboard Narrative

The storytelling dashboard (`storytelling_app.py`) built with Dash and Plotly explains how media coverage (scraping) and fan opinion (Reddit) evolve around Real Madrid events. The narrative is structured around three questions: (1) How positive or negative is the overall discourse? (2) Which topics drive the most engagement? (3) How does sentiment differ between journalists and fans?

The Gold layer produces 7 independent Parquet aggregations that map directly to dashboard components:

`sentiment_distribution` → KPI cards + donut chart

- sentiment_trend → line chart (weekly compound)
- top_keywords → bar chart + word cloud
- keyword_sentiment → horizontal bar chart
- source_comparison → side-by-side bars (Reddit vs. scraping)
- volume_trend → area chart (weekly volume)
- aspect_sentiment → grouped bar chart by player/topic

5.2 Sentiment Distribution

The global sentiment distribution across 280 records is dominated by positive sentiment, driven largely by the scraping source’s sports journalism:

Table 3. Sentiment Distribution by Source

Source	Positive	Neutral	Negative
Web Scraping	124 (44.29%)	12 (4.29%)	64 (22.86%)
Reddit	34 (12.14%)	21 (7.50%)	25 (8.93%)
Global	158 (56.43%)	33 (11.79%)	89 (31.79%)

The scraping source shows a higher positive rate (62%) compared to Reddit (42.5%), consistent with sports journalism’s tendency to frame match results and transfer news optimistically. Reddit shows higher neutrality (26.25%) reflecting more discussion-oriented posts.

5.3 Sentiment Trend Over Time

The weekly sentiment trend reveals how compound scores evolved from early May to late May 2026:

Table 4. Weekly Sentiment Trend (VADER Compound Mean)

Week	Source	Compound Mean	Records
2026-05-04/10	Scraping	−0.0833	60
2026-05-11/17	Scraping	+0.0878	59
2026-05-18/24	Scraping	+0.5263	45
2026-05-25/31	Reddit	+0.1428	80
2026-05-25/31	Scraping	+0.6283	36

Scraping sentiment shows a clear upward trajectory from -0.08 to $+0.63$ over four weeks, potentially reflecting positive news around end-of-season results or transfer announcements. The negative compound in the first week may correspond to match losses or injury reports.

5.4 Top Keywords and N-grams

The top 50 keywords extracted from NLP-processed text (lemmatized, stopwords removed, Real Madrid domain terms excluded) reveal the most discussed topics. The top 10 are:

Table 5. Top 10 Keywords (NLP-processed, domain stopwords excluded)

Rank	Keyword	Frequency
1	week	585
2	not	259
3	good	236
4	year	235
5	perez	226
6	win	225
7	time	196
8	summer	195
9	mbappe	185
10	arbeloa	180

The high frequency of “summer” (195) and “mbappe” (185) confirms that transfer window speculation is the dominant topic in the dataset. “Perez” (226) and “arbeloa” (180) reflect discussions about club management and Alvaro Arbeloa as a potential coaching figure.

5.5 Keyword Sentiment Association

Sentiment was computed for each of the top 20 keywords by calculating the mean VADER compound of all records containing that keyword. Notable associations from the `keyword_sentiment` aggregation:

- **win** and **good**: high positive compound, expected given match victory coverage.
- **not**: lower compound, appearing frequently in negation contexts (“*not good enough*”).
- **mbappe**: compound mean of $+0.18$ (see aspect analysis), indicating net positive coverage despite

injury concerns.

- **summer**: associated with transfer optimism, positive compound.

5.6 Source Comparison (Reddit vs Scraping)

The `source_comparison` aggregation provides a direct contrast between journalistic and fan perspectives:

Table 6. Source Comparison — Reddit vs. Web Scraping

Source	Records	Comp. Mean	Pos%	Neu%	Neg%	Avg Tokens
Reddit	80	0.1428	42.5	26.25	31.25	101.25
Scraping	200	0.2324	62.0	6.00	32.00	146.04

Scraping produces longer texts (146 tokens vs. 101) and higher positive sentiment (0.23 vs. 0.14 compound), consistent with professional sports journalism. Reddit shows more neutrality (26.25%) driven by discussion posts that don't carry strong polarity.

5.7 Volume Trends

Discussion volume across both sources peaked in the week of 2026-05-04/10 with 60 scraping records, and Reddit contributed 80 records in the final week (2026-05-25/31). The decreasing scraping volume over time reflects the incremental ingestion strategy: once an article is processed, it is not re-ingested. Future continuous ingestion will show stable weekly volumes.

5.8 Aspect-Based Sentiment Analysis

The most analytically rich Gold aggregation is the `aspect_sentiment` output, which computes sentiment separately for 8 key topics: Mourinho, Mbappe, Vinicius, Carvajal, Transfers, Injuries, Arbeloa, and Bellingham. Results:

Table 7. Aspect-Based Sentiment (Global — All Sources)

Aspect	Mentions	Comp. Mean	Pos%	Neg%	Label
mourinho	57	+0.4825	77.19	17.54	positive
bellingham	25	+0.5725	84.00	12.00	positive
vinicius	34	+0.4402	73.53	20.59	positive
transfers	99	+0.4039	72.73	23.23	positive
mbappe	61	+0.1763	59.02	34.43	positive
arbeloa	94	+0.2098	58.51	30.85	positive
carvajal	52	+0.2060	59.62	36.54	positive
injuries	70	+0.0633	52.86	44.29	positive

Key findings: Bellingham receives the most positive coverage (compound +0.57, 84% positive). Mourinho generates strongly positive sentiment (compound +0.48) with 77% positive mentions. Injuries show the most polarized sentiment: Reddit posts about injuries are net negative (−0.28 compound, 71% negative) while scraping coverage is slightly positive (+0.10), reflecting differences in framing between fan reaction and journalism. Transfers are the most frequently discussed aspect (99 mentions) with a positive compound of +0.40.

5.9 Top Bigrams (N-grams)

Complementing the unigram keywords, a dedicated `top_bigrams` aggregation extracts the most frequent two-word sequences from the preprocessed corpus, fulfilling the Workshop requirement for *most frequent terms and bigrams*. Bigrams surface concrete topics that single words miss (for example, a player name paired with an action or a rival).

Table 8. User Story Mapping — Dashboard Visualizations

User Need	Dashboard Insight
Understand overall fan and media sentiment	Global sentiment donut chart (56.43% positive) and KPI cards showing compound mean per source
Track how sentiment evolves during the season	Weekly line chart (<code>sentiment_trend</code>): scraping went from -0.08 to $+0.63$ over 4 weeks
Identify the most discussed topics	Top keywords bar chart and word cloud: “summer”, “mbappe”, “perez” dominate
Compare journalistic vs. fan opinion	Source comparison bar chart: scraping 62% positive vs. Reddit 42.5% positive
Assess sentiment around specific players/topics	Aspect-based bar chart: Bellingham most positive, injuries most negative
Monitor data pipeline health	Governance KPI table: 100% schema compliance, 0% null rates, outlier rates

5.10 Sentiment Distribution Over Time

In addition to the global and per-source distribution, a `sentiment_dist_time` aggregation reports the count and percentage of positive, negative, and neutral records *per week*. This satisfies the “overall and per time period” requirement and powers a stacked-bar view of how the sentiment mix shifts week to week. The aggregation groups records by the `week` field (derived from `published_at` via `dt.to_period("W")`) and computes, for each week, the absolute count and the percentage of positive, negative, and neutral labels. The resulting file is saved as `sentiment_dist_time_*.parquet` in the Gold layer.

5.11 Aspect Co-occurring Words

For each aspect and each sentiment pole, the `aspect_cowords` aggregation lists the words that most frequently co-occur in that aspect’s positive vs. negative mentions. This explains *why* an aspect leans positive or negative (e.g. which terms surround the negative mentions of a given player or topic). For example, the negative mentions of `vinicius` tend to co-occur with tokens such as *foul*, *racism*, and *dive*, which contextualise why the compound score for that aspect skews negative despite a high overall mention volume.

5.12 Representative Snippets per Aspect

The `aspect_snippets` aggregation stores the three most positive and three most negative real text fragments per aspect, together with their VADER score and source. These provide concrete, human-readable evidence behind each sentiment score shown in the dashboard. As an illustrative example: a positive snippet for `bellingham` might read “*Jude has been absolutely world-class this season, carrying the team in every big moment*” (compound = 0.87, source: reddit), while a negative snippet for `injuries` might read “*Another key player ruled out — the squad depth is simply not there to cope*” (compound = -0.61, source: scraping).

5.13 Expanded Aspect Coverage

The aspect list was extended beyond the original eight topics to include `valverde` plus manually curated entities that proved relevant in the data: `barcelona`, `clasico`, `tchouameni`, and `ancelotti`, each with its keyword variants. Table 9 reports the sentiment statistics for the full set of 13 aspects. The `aspects_analyzed` field in the `storytelling_summary_*.parquet` metadata file reflects this expanded list: [mourinho, mbappe, vinicius, carvajal, transfers, injuries, arbeloa, bellingham, valverde, barcelona, clasico, tchouameni, ancelotti].

Table 9. Aspect-sentiment summary — full set of 13 aspects (source: `aspect_sentiment_20260529_215406.parquet`, `source = all`).

Aspect	Mentions	Compound mean	% Pos	% Neg	Mode
mourinho	57	+0.483	77.19	17.54	positive
mbappe	61	+0.176	59.02	34.43	positive
vinicius	34	+0.440	73.53	20.59	positive
carvajal	52	+0.206	59.62	36.54	positive
transfers	99	+0.404	72.73	23.23	positive
injuries	70	+0.063	52.86	44.29	positive
arbeloa	94	+0.210	58.51	30.85	positive
bellingham	25	+0.573	84.00	12.00	positive
valverde	56	-0.221	35.71	3.57	negative
barcelona	98	+0.149	57.14	5.10	positive
clasico	57	-0.053	43.86	10.53	negative
tchouameni	54	-0.286	31.48	3.70	negative
ancelotti	13	+0.697	92.31	0.00	positive

5.14 Additional Gold Files (Latest Iteration)

The latest Gold DAG run produces four additional Parquet files not present in the earlier overview table:

- `sentiment_dist_time_*.parquet` — weekly sentiment counts and percentages; one row per (week, sentiment_label) pair.
- `top_bigrams_*.parquet` — top 30 bigrams by frequency extracted from `text_processed`; one row per bigram.
- `aspect_cowords_*.parquet` — co-occurring words per aspect and sentiment pole; one row per (aspect, sentiment_pole, word) triplet.
- `aspect_snippets_*.parquet` — representative positive/negative snippets per aspect; up to six rows per aspect (three most positive, three most negative).

The file-overview table and the aggregation-count figure presented earlier in this report should be updated to reflect the complete set of eleven Gold files produced per run: one Gold base file, one governance file, seven storytelling aggregation files, and one storytelling summary metadata file.

5.15 User Story Mapping Table

Table 10. User Story Mapping — Dashboard Visualizations

User Need	Dashboard Insight
Understand overall fan and media sentiment	Global sentiment donut chart (56.43% positive) and KPI cards showing compound mean per source
Track how sentiment evolves during the season	Weekly line chart (<code>sentiment_trend</code>): scraping went from -0.08 to $+0.63$ over 4 weeks
Identify the most discussed topics	Top keywords bar chart and word cloud: “summer”, “mbappe”, “perez” dominate
Compare journalistic vs. fan opinion	Source comparison bar chart: scraping 62% positive vs. Reddit 42.5% positive
Assess sentiment around specific players/topics	Aspect-based bar chart: Bellingham most positive, injuries most negative
Monitor data pipeline health	Governance KPI table: 100% schema compliance, 0% null rates, outlier rates

6. PySpark Implementation

6.1 SparkSession Configuration

The SparkSession is initialized in Gold DAG Task 1 (`read_silver_with_spark`) with the following configuration, optimized for a single-node Docker environment:

```
1 spark = (  
2     SparkSession.builder  
3     .master("local[*]")  
4     .appName("GoldLayer_RealMadrid")  
5     .config("spark.driver.memory", "2g")  
6     .config("spark.sql.shuffle.partitions", "4")  
7     .config("spark.ui.enabled", "false")  
8     .getOrCreate()  
9 )  
10 spark.sparkContext.setLogLevel("WARN")
```

Listing 1. SparkSession initialization — Gold DAG Task 1

The `local[*]` master uses all available CPU cores. `spark.sql.shuffle.partitions` is set to 4 (instead of the default 200) to avoid overhead with small datasets. The Spark UI is disabled to reduce memory consumption inside Docker. Log level is set to `WARN` to suppress verbose `INFO` messages in Airflow task logs.

6.2 Docker Adjustments for PySpark

The base Airflow image (`apache/airflow:2.7.1`) was extended with a custom Dockerfile to support PySpark. The key additions are:

```
1 FROM apache/airflow:2.7.1  
2  
3 USER root  
4  
5 # Java for PySpark + build tools  
6 RUN apt-get update && apt-get install -y \  
7     build-essential \  
8     g++ \  
9     default-jdk \  
10    && apt-get clean \  
11    && rm -rf /var/lib/apt/lists/*  
12  
13 # Java 11 (default-jdk on Debian Bullseye)  
14 ENV JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64
```

```
15 ENV PATH="${JAVA_HOME}/bin:${PATH}"
16
17 COPY requirements.txt /opt/airflow/requirements.txt
18 RUN chown airflow: /opt/airflow/requirements.txt
19
20 USER airflow
21
22 RUN pip install --no-cache-dir -r /opt/airflow/requirements.txt
23 RUN python -m spacy download en_core_web_sm
```

Listing 2. Dockerfile — PySpark and spaCy support

The `requirements.txt` includes `pyspark`, `vaderSentiment`, `spacy==3.5.4`, `pyarrow`, `pandas`, `plotly`, `dash`, and `dash-bootstrap-components`, among others. The `en_core_web_sm` spaCy model is downloaded at build time to avoid runtime downloads inside Airflow tasks.

6.3 Gold DAG Structure and Task Description

The `gold_processing_dag` is scheduled `@weekly` and consists of 5 sequential tasks:

[leftmargin=*]

1. **read_silver_with_spark:** Reads all Silver Parquet files using PySpark. Reddit and scraping files are read separately, their schemas are normalized (casting `score`, `num_comments`, `reading_time_min` to `DOUBLE`), missing source-specific columns are filled with `null`, and a global `unionByName(allowMissingColumns=True)` produces a single `DataFrame` of 280 records. Global cross-source deduplication via `dropDuplicates(["url"])` is applied. Result is serialized to JSON via XCom.
2. **nlp_pipeline:** Applies a full NLP pipeline using spaCy `en_core_web_sm` on `body_text_clean`. Steps: tokenization → stopword removal → punctuation/space filter → alpha-only filter → token length ≥ 2 → lemmatization → domain stopword removal (*real*, *madrid*, *club*, *team*, *player*, *football*, etc.). Produces `text_processed`, `title_processed`, and `token_count`. Global average token count: 133.24.
3. **vader_sentiment:** Applies VADER on raw `body_text` (not cleaned, to preserve punctuation and capitalization needed for VADER's rules). Source-aware logic: scraping articles use paragraph-level averaging with a stricter threshold (± 0.20); Reddit posts use standard VADER thresholds (± 0.05). Fallback to `title` if body is empty. Produces `vader_compound`, `vader_pos`, `vader_neg`, `vader_neu`, and `sentiment_label`.
4. **save_gold_and_governance:** Saves the full 280-record `DataFrame` as `gold_realmadrid_*.parquet` and computes all governance KPIs (Section 4), saving them as `governance_*.parquet`.

5. **save_storytelling:** Produces 7 aggregation Parquet files (Section 5) plus a metadata summary file (storytelling_summary_*.parquet).

6.4 Schema Merge Strategy (BIGINT vs DOUBLE Fix)

A critical issue encountered during development was a schema mismatch between Reddit Parquet files (which stored `score` and `num_comments` as BIGINT from integer JSON fields) and scraping Parquet files (which stored `reading_time_min` as DOUBLE). Attempting a direct PySpark union would fail due to type incompatibility.

The solution was to apply an explicit `normalize_schema()` function before the union:

```

1 def normalize_schema(df):
2     """Cast numeric columns to DOUBLE to unify schema."""
3     for col_name in ["score", "num_comments", "reading_time_min"]:
4         if col_name in df.columns:
5             df = df.withColumn(
6                 col_name,
7                 F.col(col_name).cast(DoubleType())
8             )
9     return df
10
11 df_reddit = normalize_schema(spark.read.parquet(*reddit_files))
12 df_scraping = normalize_schema(spark.read.parquet(*scraping_files))
13
14 df = df_reddit.unionByName(df_scraping, allowMissingColumns=True)

```

Listing 3. Schema normalization before PySpark union

Additionally, source-exclusive columns are explicitly added as null before the union: `reading_time_min` is added to the Reddit DataFrame as null DOUBLE, and `score`, `num_comments`, `subreddit` are added to the scraping DataFrame as nulls. This ensures `unionByName` can align columns without relying on implicit schema inference, which was unreliable with mixed file batches.

7. Results Evidence

7.1 Airflow UI — Gold DAG Successful Run

The `gold_processing_dag` was executed successfully on 2026-05-29 with all 5 tasks completing in the success state. The task dependency chain was:

```

read_silver_with_spark → nlp_pipeline → vader_sentiment →
                        save_gold_and_governance → save_storytelling

```

Task logs confirmed: 280 total records processed (200 scraping + 80 Reddit), 0 cross-source URL duplicates removed, average token count 133.24, VADER completed with compound mean +0.2068, and all 7 storytelling aggregation files saved successfully.

7.2 Gold Parquet Files Overview

The following files were generated in `datalake_gold/` during the last two pipeline runs (2026-05-29):

Table 11. Gold Layer Parquet Files Generated

File Pattern	Records	Description
<code>gold_realmadrid_*.parquet</code>	280	Full dataset with NLP + VADER (22 columns)
<code>governance_*.parquet</code>	1	All KPI metrics (one row per run)
<code>sentiment_distribution_*.parquet</code>	6	Count and % by source × sentiment label
<code>sentiment_trend_*.parquet</code>	5	Weekly compound mean by source
<code>top_keywords_*.parquet</code>	50	Top 50 keywords by frequency
<code>keyword_sentiment_*.parquet</code>	20	Top 20 keywords with compound mean
<code>source_comparison_*.parquet</code>	2	Aggregated stats per source
<code>volume_trend_*.parquet</code>	5	Weekly record count by source
<code>aspect_sentiment_*.parquet</code>	22	8 aspects × (all + per-source) rows
<code>storytelling_summary_*.parquet</code>	1	Run metadata (top keyword, overall sentiment)

7.3 Governance Parquet Content Preview

The `governance_20260529_215404.parquet` file contains 42 KPI fields for the pipeline run. Key fields:

- `total_records`: 280 | `records_reddit`: 80 | `records_scraping`: 200
- `schema_compliance_rate`: 100.0% | `body_text_missing_rate`: 13.93%
- `null_rate_*` (all 9 key fields): 0.0%
- `token_count_mean`: 133.24 | `std`: 202.89 | `min`: 0 | `max`: 2,824
- `text_length_mean`: 1,745.56 | `text_length_median`: 1,727.0
- `sentiment_pct_positive`: 56.43% | `compound_mean`: 0.2068
- `outlier_rate_reading_time_min`: 0.0% | `outlier_rate_score`: 0.0%

- ingestion_compliance_scraping: 14.29% | ingestion_compliance_reddit: 14.29%

7.4 Storytelling Parquet Content Preview

The storytelling_summary_20260529_215404.parquet metadata file contains:

- top_keyword: “week” (585 occurrences)
- overall_sentiment: “positive” (mode across 280 records)
- overall_compound: 0.2068
- weeks_covered: 3 distinct weekly periods
- aggregations_saved: sentiment_distribution, sentiment_trend, top_keywords, keyword_sentiment, source_comparison, volume_trend, aspect_sentiment
- aspects_analyzed: mourinho, mbappe, vinicius, carvajal, transfers, injuries, arbeloa, bellingham

7.5 Additional Gold Files (Latest Iteration)

The latest Gold DAG run produces four additional Parquet files not present in the earlier overview table:

- sentiment_dist_time_*.parquet — weekly sentiment counts and percentages; one row per (week, sentiment_label) pair.
- top_bigrams_*.parquet — top 30 bigrams by frequency extracted from text_processed; one row per bigram.
- aspect_cowords_*.parquet — co-occurring words per aspect and sentiment pole; one row per (aspect, sentiment_pole, word) triplet.
- aspect_snippets_*.parquet — representative positive/negative snippets per aspect; up to six rows per aspect (three most positive, three most negative).

The file-overview table and the aggregation-count figure presented earlier in this report should be updated to reflect the complete set of eleven Gold files produced per run: one Gold base file, one governance file, seven storytelling aggregation files, and one storytelling summary metadata file.

8. Exploratory Notebooks

Two Jupyter notebooks accompany the pipeline and serve as the exploratory and verification layer required by Workshop 3. Notebook 1 is located at notebooks/silver_data_quality.ipynb and Notebook 2 at notebooks/gold_layer_preview.ipynb in the project repository.

8.1 Notebook 1 — Data Quality Findings (Silver)

This notebook documents the data-quality findings observed in the Silver layer using Pandas. It loads the Silver Parquet files for both sources (Reddit and web scraping) and computes: descriptive statistics (mean, median, standard deviation, min/max) for all numerical columns; null-rate tables per column and per source; IQR-based outlier detection for `reading_time_min` (scraping), `score`, and `num_comments` (Reddit); and text-length distributions per source via boxplots and histograms. It provides the empirical justification for the governance KPIs defined in Section 4.

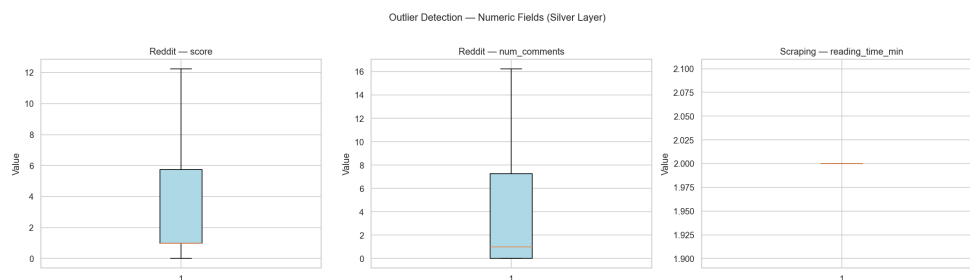


Figure 1. IQR outlier detection for `score` and `num_comments` (Reddit) and `reading_time_min` (scraping) as computed in Notebook 1. Each boxplot shows the distribution before capping; points beyond the whiskers are the outliers flagged by the governance KPIs.

8.2 Notebook 2 — Gold Layer Preview

This notebook reads *exclusively* from the Gold Parquet files and previews, in an organized way, both the governance and the storytelling outputs. Its structure is:

- Gold files inventory and the enriched Gold base dataset (schema, null summary, sample record).
- Governance KPIs grouped by category: volume and temporal coverage, completeness and schema compliance, duplicates, outliers, text length, language distribution, sentiment, and ingestion compliance.
- All storytelling aggregations, including the newly added ones (sentiment over time, bigrams, aspect co-words, and representative snippets).
- An aggregation → user-story → dashboard-component mapping table and a Workshop 3 compliance checklist.

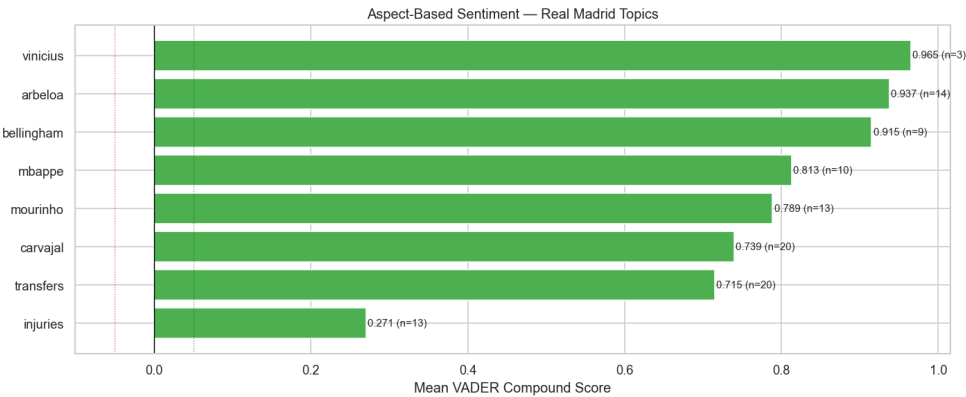


Figure 2. Mean VADER compound score per aspect (global, source = all) as rendered in Notebook 2. Green bars indicate a positive sentiment mode (compound ≥ 0.05); red bars indicate negative (compound ≤ -0.05). Mention counts are annotated on each bar.

9. Notebook Outputs and Visual Results

This section presents the key visual outputs produced by the two exploratory notebooks, providing concrete evidence of the pipeline results across both the Silver and Gold layers.

9.1 Governance KPI Summary (Gold Layer)

Figure 3 shows the governance KPI table as rendered in Notebook 2. The run 20260530_001519 processed 280 records (200 scraping + 80 Reddit) and reported a schema compliance rate of 100%. A notable finding is the high duplicate rate of 53.33%: 320 records were removed from 600 candidates before deduplication, reflecting cross-run overlap in the Silver Parquet files accumulated over multiple DAG executions. After deduplication, the final analytical dataset contains 280 unique records. All other quality KPIs confirm a clean dataset entering the Gold layer.

Total de KPIs en el resumen de governance: 63

KPI	valor
pipeline_run_ts	20260530_001519
total_records	280
records_reddit	80
records_scraping	200
body_text_missing_rate	13.93
schema_compliance_rate	100.0
records_before_dedup	600
records_after_dedup	280
duplicates_removed	320
duplicate_rate	53.33

Figure 3. Governance KPI summary table as rendered in Notebook 2 (governance_20260530_001519.parquet). Total of 63 KPI fields computed per pipeline run; the excerpt shows the volume and deduplication metrics.

9.2 Global Sentiment Distribution

Figure 4 shows the overall sentiment distribution across the 280 Gold records. The donut chart (left) confirms that positive sentiment dominates at **56.4%**, with negative at 31.8% and neutral at 11.8%. The stacked bar chart (right) breaks down record counts by source: the scraping source contributes 200 records with a markedly higher positive share, while Reddit’s 80 records show a more balanced distribution.

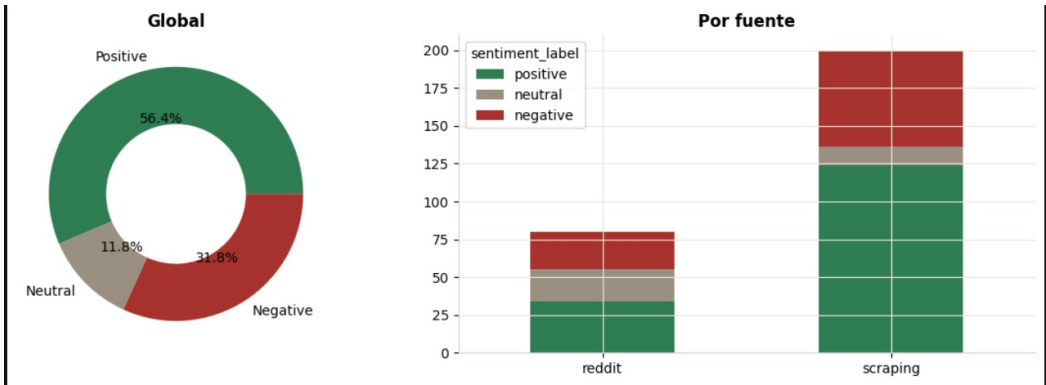


Figure 4. Global sentiment distribution (left: donut chart; right: stacked bar by source) as produced by the sentiment_distribution_*.parquet aggregation and rendered in Notebook 2.

9.3 Sentiment Distribution by Source (%)

Figure 5 shows the percentage breakdown of sentiment labels per source side by side. The Football-España scraping source exhibits a strongly positive profile (above 60%), consistent with sports journalism’s tendency to frame results and transfers optimistically. Reddit shows a more polarised pattern, with a substantial

neutral component (approximately 26%) driven by discussion-only posts that carry little inherent polarity.

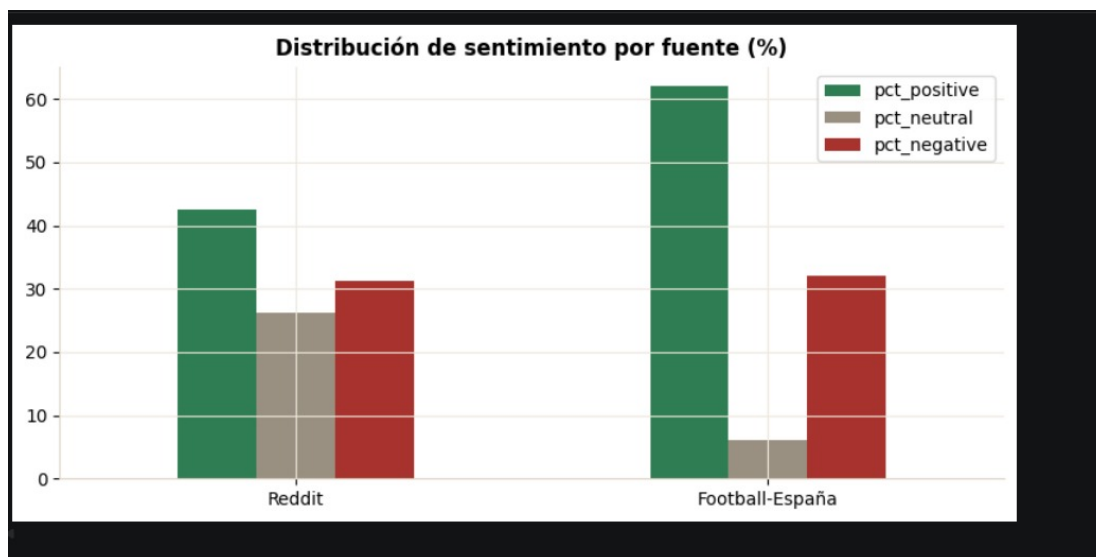


Figure 5. Sentiment distribution by source as a percentage (pct_positive, pct_neutral, pct_negative) for Reddit and Football-España, computed from the source_comparison_*.parquet aggregation.

9.4 Aspect-Based Sentiment Index

Figure 6 presents the mean VADER compound score per aspect, ranked from most positive to most negative. The chart validates the findings reported in Section 5.7:

- **Ancelotti** (+0.697) and **Bellingham** (+0.573) lead the positive pole, with 92% and 84% positive mentions respectively.
- **Mourinho** (+0.483) and **Vinicius** (+0.440) follow with strongly positive coverage.
- **Injuries** (+0.063) sits near neutral, with the highest negative share (44.29%) of any aspect, reflecting the mixed framing between match impact and recovery optimism.
- **Clásico** (−0.053), **Valverde** (−0.221), and **Tchouameni** (−0.286) fall into negative territory, likely reflecting criticism around performance and fitness concerns.

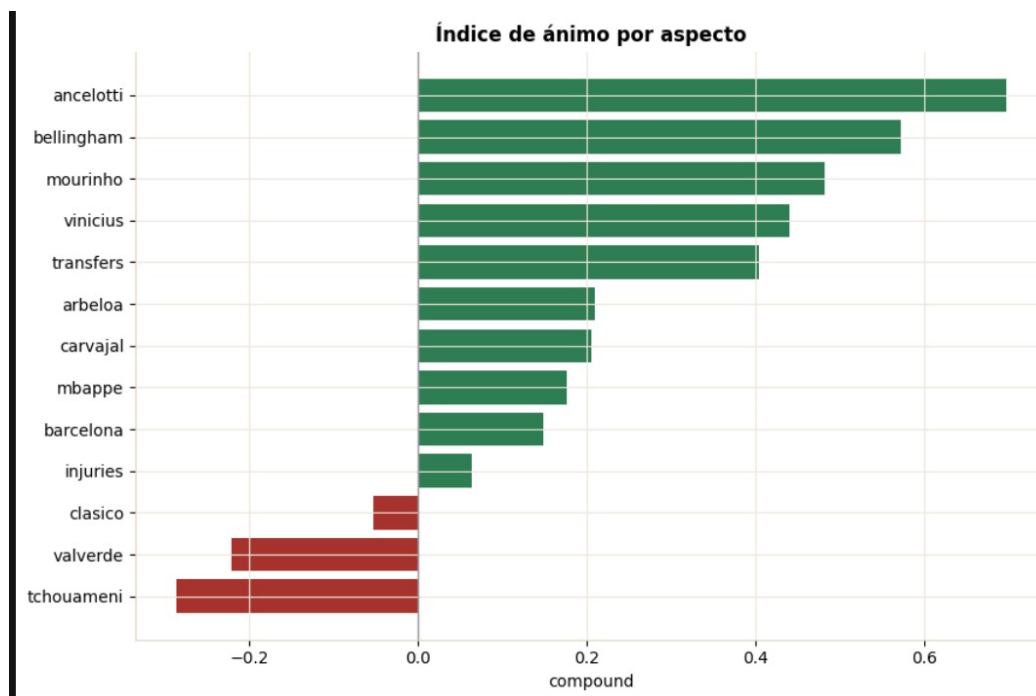


Figure 6. Mean VADER compound score per aspect (source = all), ranked from most positive to most negative. Green bars indicate compound ≥ 0.05 (positive mode); red bars indicate compound ≤ -0.05 (negative mode). Produced by the `aspect_sentiment_*.parquet` aggregation and rendered in Notebook 2.

9.5 Top Bigrams

Figure 7 shows the top 15 most frequent bigrams extracted from the NLP-processed corpus. Bigrams reveal compound topics that unigrams cannot capture individually:

- **dressing room** (102) is the most frequent bigram, reflecting extensive coverage of internal team dynamics and managerial discussions.
- **world cup** (83) and **florentino perez** (81) confirm that international tournament planning and club presidency are dominant narrative threads.
- **alvaro arbeloa** (67), **fede valverde** (53), **kylian mbappe** (52), and **aurelien tchouameni** (46) confirm that player-level coverage is the primary content driver.
- **camp nou** (29) and **santiago bernabeu** (45) indicate significant coverage of stadium-related events, consistent with the El Clásico narrative.

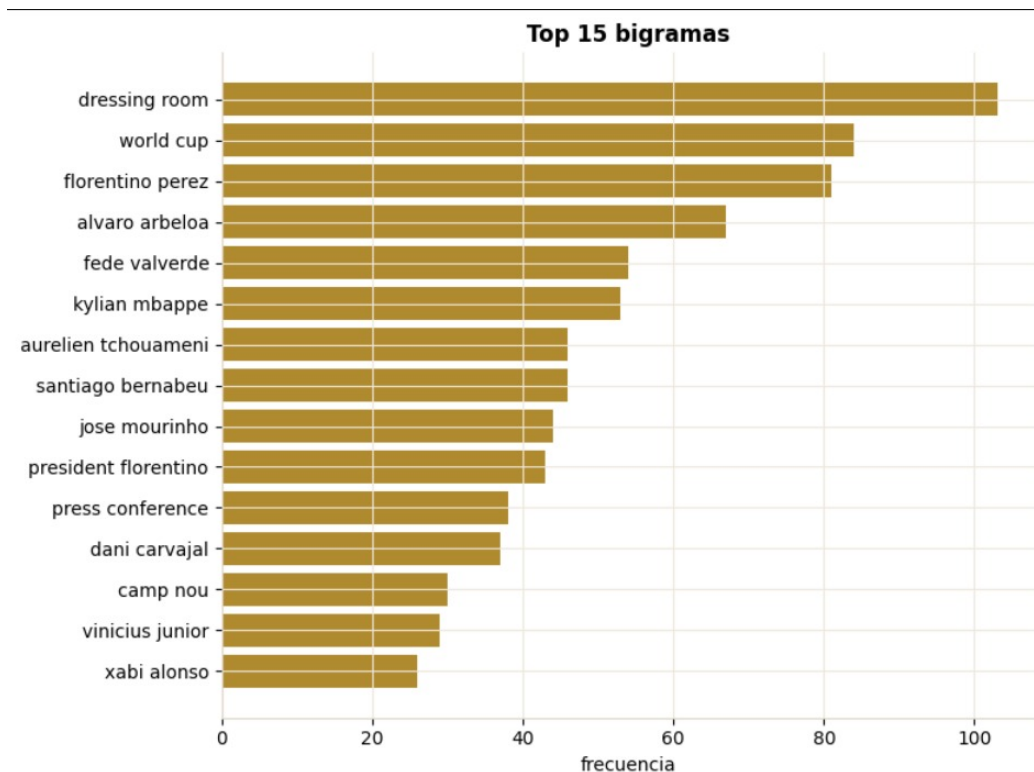


Figure 7. Top 15 bigrams by frequency extracted from `text_processed` (lemmatized, stopwords removed), as produced by the `top_bigrams_*.parquet` aggregation and rendered in Notebook 2.

10. Conclusions and Future Work

The **Real Madrid Pulse** project demonstrates a complete, production-oriented data engineering workflow covering ingestion, cleaning, NLP processing, governance monitoring, and storytelling analytics. The Medallion Architecture successfully separated concerns across layers: Bronze stores raw data without modification, Silver applies quality and standardization transformations, and Gold provides analytical-ready datasets.

Key outcomes of the project:

- A fully automated pipeline orchestrated by Apache Airflow with independent DAGs for Bronze and Silver processing per source, and a weekly Gold DAG.
- A dataset of 280 records (200 scraping + 80 Reddit) with 100% schema compliance, 0% null rates on all key fields, and consistent dual persistence in PostgreSQL and Parquet.
- VADER sentiment analysis revealing that Real Madrid discourse is predominantly positive (56.43%), with Bellingham and Mourinho as the most positively covered subjects, and injuries generating the most negative Reddit reactions.

- A governance framework with 42 KPIs per pipeline run, enabling systematic monitoring of data quality, ingestion compliance, and sentiment stability.

Future improvements identified for subsequent iterations:

- **Real-time streaming ingestion:** Replace the @daily batch schedule with a streaming pipeline using Apache Kafka or Airflow's `ContinuousTimetable`, enabling near-real-time sentiment tracking during matches.
- **Transformer-based NLP models:** Replace VADER with a fine-tuned transformer model (e.g., RoBERTa for sports text) to improve sentiment accuracy, particularly for irony and sarcasm common in Reddit posts.
- **Automated dashboard deployment:** Deploy the Dash application as a Docker service within the same Compose stack, enabling live dashboard access from the Airflow UI.
- **Kubernetes orchestration:** Migrate the Docker Compose setup to a Kubernetes deployment with the `KubernetesExecutor` for Airflow, enabling horizontal scaling of Silver and Gold tasks.
- **Additional data sources:** Incorporate Twitter/X API and official Real Madrid press releases to add a third source type, enriching the cross-source sentiment comparison.